



אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

משחק מבוך

סמסטר ב'
תשפ"ו

תקציר

- חלק א' - יצירת ספריית קוד
 - מפסאודו-קוד לתכנות מונחה עצמים
 - חלק ב' – שרת-לקוח
 - Streaming, קבצים, decorator I
 - תכנות מקבילי עם java threads
- חלק ג'
 - MVVM
 - תכנות מונחה אירועים
 - GUI בטכנולוגיית JavaFX

Table of Contents

3	מנהלות.....
4	הנחיות כלליות.....
5	הוראות הגשה.....
6	בדיקת הפרויקט.....
6	הקדמה.....
7	מבוך דו מימדי.....
8	חלק א': מפסאודו קוד לתכנות מונחה עצמים.....
9	משימה א' – אלגוריתם ליצירת מבוך.....
11	בדיקות.....
12	משימה ב' – אלגוריתמי חיפוש.....
13	בדיקות.....
14	משימה ג' – 3D-Maze.....
16	משימה ד' – Unit Testing (למידה עצמית).....
16	משימה ה' – עבודה עם מנהל גרסאות (למידה עצמית).....
18	דגשים להגשה.....
19	חלק ב': עבודה עם Streams ו-Threads.....
19	הקדמה.....
19	משימה א' – דחיסה של Maze ו- Decorator Design Pattern.....
22	בדיקות.....
23	משימה ב' – שרת-לקוח ו- Threads.....
24	בדיקות.....
26	משימה ג' – קובץ הגדרות (לימוד עצמי).....
27	דגשים להגשה.....
28	חלק ג': אפליקצית Desktop בארכיטקטורת MVVM, תכנות מונחה אירועים, ו- GUI.....
28	משימה א' – ארכיטקטורת MVVM.....
29	משימה ב' – תכנות מונחה אירועים ו- GUI.....
29	יצירת רכיב גרפים - תזכורת.....
30	כתיבת ה- GUI.....
30	עיצוב ה- GUI.....
32	משימה ג' – Maven & Log4j2.....
32	דגשים להגשה.....

מנהלות

בקורס זה יינתן תרגיל חימום אחד עם חובת הגשה וללא ציון על מנת להכין אתכם לפרויקט. התרגולים במעבדה נועדו ללמד אתכם את יסודות שפת Java, את מה שיש לה להציע ואת דרכי העבודה המומלצות יחד עם הדגמת החומר הנלמד בהרצאה. הכלים והידע שתלמדו במעבדה נדרשים לכתיבת הפרויקט וישמשו אתכם רבות בהמשך דרככם.

לפרויקט שלוש אבני דרך, מועדי הגשתם המעודכנים רשומים באתר ה Moodle של הקורס.

משקל כל חלק בפרויקט 33% ממשקל הפרויקט הכולל.

כל חלקי העבודה הינם להגשה בזוגות.

כל חלק בפרויקט נבנה בהתבסס על החלק שקדם לו לכן חשוב לעמוד בדרישות בצורה הטובה ביותר. עמידה בדרישות בחלק מסוים בפרויקט יקל עליכם בחלקים שיבואו אחריו.

הפרויקט מאורגן בצורה מרווחת מאוד כך שיש מספיק זמן לביצוע כל מטלה. כבר ניתנו הזמנים המקסימליים עבור כל מטלה, כל בקשה להארכה קולקטיבית משמעותה לבוא על חשבון המטלות האחרות. להארכות זמן פרטניות מסיבות מוצדקות בלבד יש לפנות למתרגל הקורס. **ציון "התרגול" יורכב ממוצע המטלות ומשקלו 20% מהציון הסופי בקורס. משקל הבחינה 80% .**

לקורס זה קיים מבחן תכנותי. מועדי א' ו ב' מתקיימים בתקופת הבחינות המתאימה. **קיימת חובת מעבר מבחן כדי לעבור את הקורס.** רק מי שהשקיע בעצמו בפרויקט והפנים את החומר הנלמד בקורס יוכל לעבור את המבחן.

בהצלחה,

דודי, איתן וניתאי

הנחיות כלליות

אנו ממליצים בחום לקרוא את מסמך זה מתחילתו ועד סופו ולהבין את הכיוון הכללי של הפרויקט עוד בטרם התחלתם לממש. הבנת הדרישות ומחשבה מספקת לפני תחילת המימוש תחסוך לכם זמן עקב טעויות מיותרות.

מומלץ שתעבדו עם **IntelliJ** בגרסה העדכנית ביותר. במודל יש קישור לקבלת רישיון סטודנט חינמי עבור גרסת ה-**Ultimate**. בנוסף, אם תרצו תוכלו במקום זאת להוריד את גרסת ה-**Community** החינמית, מכאן:

<https://www.jetbrains.com/idea/download/>

עבור כל חלק בפרויקט זה, שעליו אתם מתחילים לעבוד אנו ממליצים:

1. לקרוא את כל הדרישות של החלק מתחילתו ועד סופו.
2. לדון על הדרישות עם בן/בת הזוג למטלה.
3. לחשוב איך אתם הולכים לממש את הדרישות.
4. לייצר לעצמכם **Class Diagram**.
5. לתכנן את חלוקת העבודה ביניכם.
6. להתחיל לממש.
7. לשמור ולגבות את הקוד שלכם במהלך העבודה במספר מוקדים שונים: Email ו-Dropbox.
8. לבצע בדיקות עבור כל קוד שמימשתם. בדקו כל מתודה על מנת להבטיח שהיא מבצעת כראוי מה שהיא אמורה לבצע. זכרו שאתם יכולים לדבג (Debug) את הקוד שלכם ולראות מה קורה בזמן ריצה.
9. במידה ועשיתם שינויים ו"הרסתם" משהו בפרויקט, זכרו שב- IntelliJ קיימת האופציה לצפות בהיסטוריה של כל מחלקה ולעקוב אחרי שינויים (מקש ימני על קוד מחלקה ← **Local History**).
10. לשמור את התוצאה הסופית שאותה אתם הולכים להגיש במספר מוקדים שונים.

דגשים לכתיבת קוד:

- הקפדה על עקרונות ה- **SOLID** שלמדתם.
- הפונקציות צריכות להיות קצרות, עד 30 שורות ולעסוק בעניין אחד בלבד.
- פונקציות ארוכות ומסובכות שעוסקות בכמה עניינים הם דוגמא לתכנות גרוע.
- הפונקציות צריכות להיות גנריות (כלליות), שאינן תפורות למקרה ספציפי.
- שמות משתנים ברורים ובעלי משמעות.
- שמות שיטות ברורים ובעלי משמעות.
- מתן הרשאות מתאימות למשתנים ולמתודות - **Private, Protected, Public**. כימוס נכון (**Encapsulation**)¹.
- שימוש נכון בתבניות העיצוב שנלמדו בכיתה, בירושה ובממשקים.
- תיעוד הקוד:
 - תיעוד מעל מחלקות, שיטות וממשקים.
 - יש לתעד שורות חשובות בתוך המימוש של השיטות.
 - הסבר על תיעוד באמצעות Javadoc ניתן למצוא [בקישור הזה](#).

בסיום כתיבת הפתרון:

1. הריצו את הפרויקט ובדקו אותו ע"פ הדרישות של החלק אותו מימשתם.
2. עברו שוב על דרישות המטלה ובדקו שלא פספסתם אף דרישה.
3. חשבו על מצבי קצה שאולי עלולים לגרום לאפליקציה שלכם לקרוס וטפלו בהם.

קחו בחשבון שהפרויקט שאתם מגישים נבדק על מחשב אחר מהמחשב שבו כתבתם את הקוד שלכם. לכן, אין להניח שקיים כוון מסוים (לדוגמא: D) או תיקיות אחרות בעת ביצוע קריאה וכתובה מהדיסק.

¹ <https://he.wikipedia.org/wiki/%D7%9B%D7%99%D7%9E%D7%95%D7%A1>

בנוסף, כאשר אתם כותבים ממשק משתמש, בין אם זה **Console** או **GUI**, קחו בחשבון שהמשתמש או הבודק אינו תמיד מבין מה עליו לעשות ולכן עלול לבצע מהלכים "לא הגיוניים" בניסיונו להבין את הממשק שלכם. מהלכים אלו עלולים להביא לקריסה של הקוד אם לא עשיתם בדיקות על הקלט

שהמשתמש הכניס או לא לקחתם בחשבון תרחישים מסוימים. לכן, חשוב שתעזרו למשתמש/בודק לעבוד מול הממשק שלכם ע"י הדפסה של הוראות ברורות מה עליו להקליד, על איזה כפתור ללחוץ מתי ומה האופציות שעומדות בפניו. הנחיות אלו יפחיתו את הסיכויים לטעויות משתמש שעלולות לגרום לקריסה של האפליקציה ולהורדת נקודות.

חלקי הפרויקט שאתם מגישים נבדקים אוטומטית (פרט לחלק ג' שייבדק גם ידנית) ע"י קוד בדיקה לכן חשוב ביותר להיצמד להוראות ולוודא ששמות המחלקות והממשקים שהגדרתם הם בדיוק מה שהתבקשתם. הבדיקה האוטומטית אינה סלחנית לכן השתדלו להימנע מטעויות מצערות.

לפני הגשת המטלות, בדקו את עצמכם שוב! פעמים רבות שינויים פזיזים של הרגע האחרון שנעשים ללא מחשבה מספקת גורמים לשגיאות ולהורדת נקודות. חבל לאבד נקודות בגלל טעויות של הרגע האחרון.

הוראות הגשה

את המטלות יש להגיש למודל בתיבת ההגשה המתאימה.

- בחרו סטודנט אחד מתוך הצוות שדרך חשבונו תעלו את העבודות.
- למען הסדר הטוב, אתם מתבקשים להעלות את העבודה רק דרך החשבון של הסטודנט הנבחר (ולא מחשבונות שונים בכל פעם).
- המטלות יכתבו ב- Java Language Level 15, ובחלק ג- בשימוש עם Javafx 16, בלבד!

אתם מגישים למקורות לעי"ל את התוצרים כקובץ **Zip** המכיל:

1. תיקיית הפרויקט שלכם במלואה! התיקייה צריכה להכיל:

a. תיקיית **src** קוד המקור.

b. תיקיות נוספות כגון **resources**.

c. תיקיית ה-idea.

d. וכו'...

2. קובץ ה-**JAR** של הפרויקט שלכם.

a. הוראות יצירה ב- IntelliJ [בקישור הזה](#).

- בתיקייה שאתם מגישים מחקו קבצים ותיקיות מיותרות כגון קבצים זמניים או קבצים השייכים ל-Git. (תיקייה נסתרת), התיקייה המוגשת צריכה להיות רזה יחסית.
- במידה ואתם משתמשים הקבצי מדיה, בחרו את הקבצים הרזים ביותר (מבחינת גודל קובץ) המספיקים לכם.

קובץ ה zip יכיל את תעודות הזהות של המגישים בפורמט: ID1_ID2.zip. לדוגמה: 012345678_123456789.zip

חשוב להגיש את המטלות בזמן, מטלות שיוגשו לאחר המועד ללא הצדקה לא יבדקו.

הפרויקט המוגש לבדיקה צריך:

- להתקמפל ללא שגיאות. פתרון שאינו מתקמפל כלל לא ייבדק וציונו יהיה 0.
- לרוץ ולבצע את מה שהתבקש ללא שגיאות בזמן ריצה. כל חריגה (Exception) שתגרום לקריסה של האפליקציה בזמן הריצה תגרום להורדה של נקודות.

בדיקה עצמית של הקוד בחלק א' + חלק ב':

על מנת לבדוק את הקוד אשר כתבתם, נצרף לכם קובץ main שמטרתו העיקרית לראות שאין שגיאות מהותיות. אם עברתם את כל הטסטים בקובץ זה אין זה אומר שקיבלתם 100, הבדיקות שעליהן תקבלו ציון יהיו שונות. נא לא להגיש את קובץ main אשר תקבלו, הגשת קובץ זה יכול להכשיל את הטסטים שלכם.

בדיקת הפרויקט

בודק התרגילים בקורס זה הוא ניתאי יעקובי - yakoby@post.bgu.ac.il

עם פרסום הציונים יפורסם גם מפתח בדיקה שיפרט את הניקוד עבור כל חלק שנבדק.

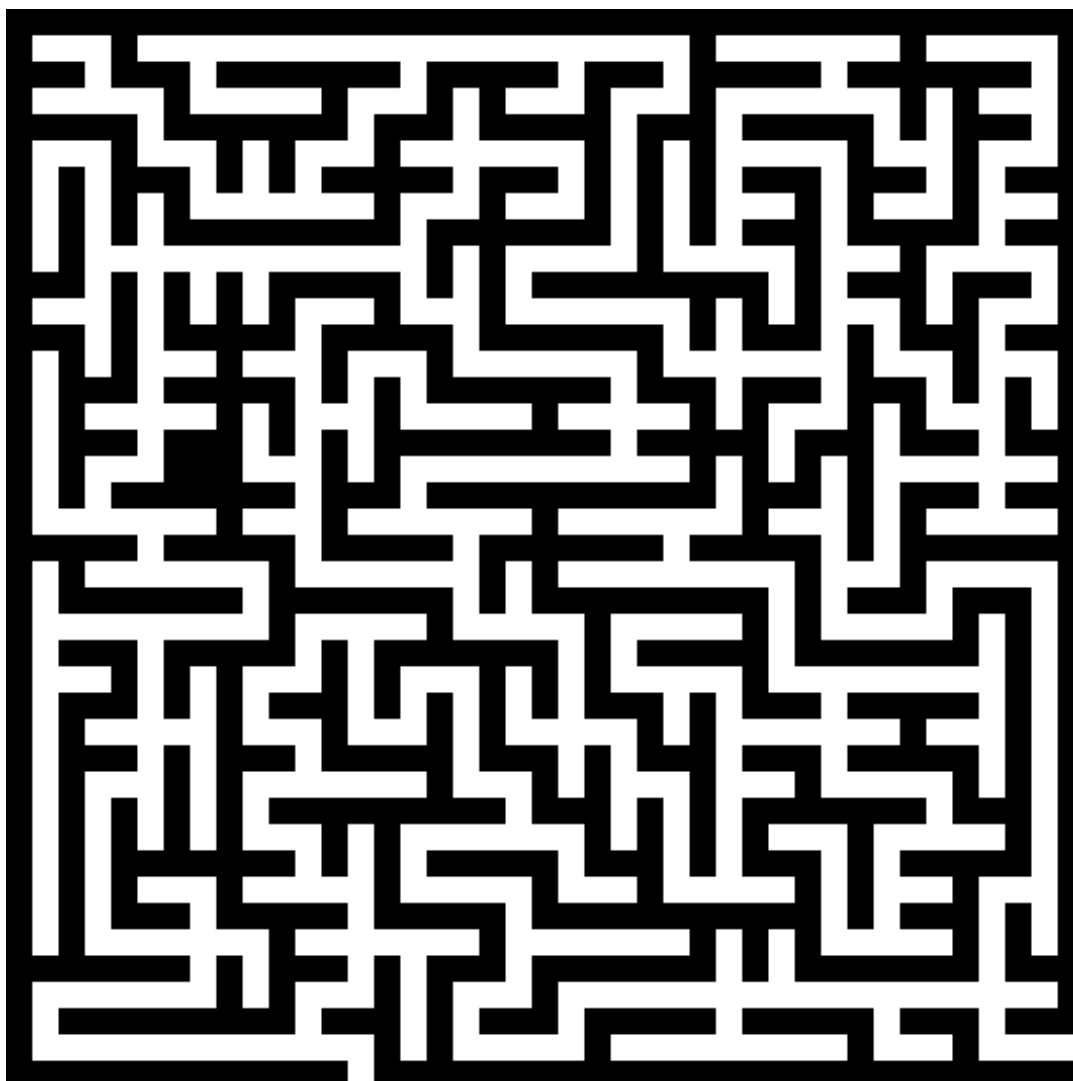
כפי שפורט קודם לכן, העבודות נבדקות אוטומטית ע"י קוד בדיקה כך שאין מקום לערעורים מאחר והבדיקה זהה לכולם. **במקרים מיוחדים, ניתן להגיש ערעור עד 3 ימים ממועד פרסום הציונים.** פניות שיוגשו לאחר מכן לא יבדקו. הגשת ערעור תתבצע במייל בלבד ותכיל בכותרת המייל את השמות ות"ז של הסטודנטים. **שימו לב: הגשת ערעור תגרור בדיקה מחודשת ויסודית של העבודה ועלולה אף להוביל להורדה נוספת של נקודות,** לכן אל תקלו ראש ותגישו ערעורים על זוטות.

אין אפשרות לבדיקה חוזרת של מטלות. במקרים מיוחדים של טעויות קריטיות שהובילו להורדת ניקוד נרחבת עקב אי-יכולת לבדוק את המטלה תתאפשר בדיקה חוזרת אך יורדו 20 נקודות.

בהצלחה!

הקדמה - מבוך

מבוך הוא חידה הבנויה ממעברים מתפצלים, אשר על הפותר למצוא נתיב דרכה, מנקודת הכניסה לנקודת היציאה. הדמות יכולה לנוע ימינה, שמאלה, למעלה או למטה, במידה והיעד פנוי מקיר כמובן.



דוגמא למבוך

ייצוג המבוך

את המבוך עליכם לייצג כמערך דו-מימדי של `int`. הערך 1 מסמן תא מלא (קיר) ואילו 0 מסמן תא ריק. נקודת הכניסה לצורך הדוגמה מסומנת באדום ונקודת היציאה בירוק.

לדוגמא:

```
int[] [] maze={  
    {0,0,1,0,1,0,0,0,1},  
    {1,0,1,0,1,0,1,0,0},  
    {1,0,0,0,0,0,0,1,1},  
    {1,0,1,1,0,1,0,1,1},  
    {0,0,1,1,0,1,0,1,1},  
    {1,1,0,0,0,1,0,0,0},  
};
```

המבוך בעל שני מימדים, נקרא להם `rows` ו-`columns` המייצגים את מספר השורות והעמודות במבוך. אין חובה ש-`rows=columns`, כלומר שהמבוך אינו בהכרח ריבוע, הוא יכול להיות גם מלבן. בנוסף, אין חובה שלמבוך תהיה מסגרת חוסמת ונקודות יציאה מהמסגרת, כפי שמופיע בדוגמה.

חלק א': מפסאודו-קוד לתכנות מונחה עצמים

בשיעורים האחרונים למדנו כיצד לתרגם פסאודו-קוד של אלגוריתם לתכנות מונחה עצמים.

למדנו שני כללים חשובים:

כלל ראשון: להפריד את האלגוריתם מהבעיה שאותה הוא פותר.

כשנתבונן בפסאודו-קוד של האלגוריתם נסמן את השורות שהן תלויות בבעיה. שורות אלה יגדירו לנו את הפונקציונליות הנדרשת מהגדרת הבעיה. את הפונקציונליות הזו נגדיר בממשק מיוחד עבור הבעיה הכללית. מאוחר יותר מחלקות קונקרטיות יממשו את הממשק הזה ובכך יגדירו בעיות ספציפיות שונות.

שמירה על כלל זה תאפשר לאלגוריתם לעבוד מול טיפוס הממשק במקום מול טיפוס של מחלקה ספציפית. תכונת הפולימורפיזם תאפשר לנו להחליף מימושים שונים לבעיה מבלי שנצטרך לשנות דבר בקוד של האלגוריתם. **על אילו מעקרונות SOLID שמרנו כאן?**



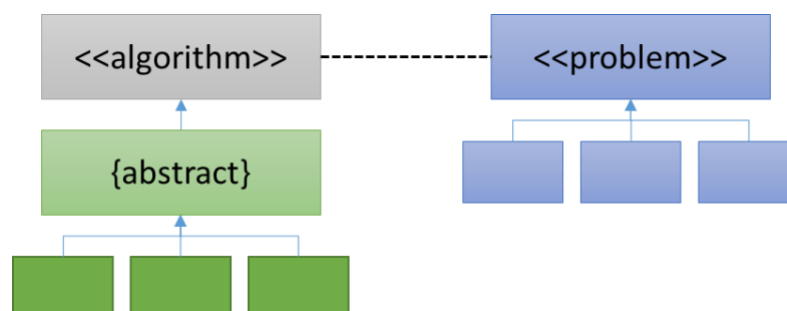
כלל שני: לממש את האלגוריתם באמצעות היררכיית מחלקות.

ייתכנו אלגוריתמים נוספים שנצטרך לממש בעתיד, או מימושים שונים לאותו האלגוריתם שלנו. לכן כבר עכשיו ניצור היררכיית מחלקות שבה:

- את הפונקציונליות של האלגוריתם נגדיר בממשק משלו.
- את מה שמשותף למימושים השונים נממש במחלקה אבסטרקטית.
 - את מה שלא משותף – נשאיר אבסטרקטי.
- את המימושים השונים ניצור במחלקות שירשו את המחלקה האבסטרקטית.
 - הם יצטרכו לממש רק את מה ששונה בין האלגוריתמים.

את ההיררכיה הזו ניתן כמובן להרחיב ע"פ הצורך.

קבלנו את המבנה הבא:



על אילו מעקרונות SOLID שמרנו כאן?

משימה א' – אלגוריתם ליצירת מבוך

ניקוד החלק: 45 נקודות.

צרו פרויקט בשם ATP-Project-PartA ובתוכו חבילה (Package) בשם algorithms.mazeGenerators. כלומר, חבילה בשם algorithms ובתוכה חבילה בשם mazeGenerators.

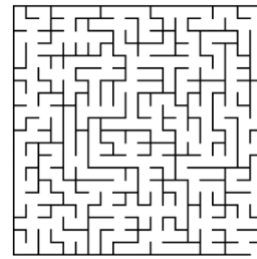
בפנים צרו מחלקה בשם Maze המייצגת מבוך כבתיאור לעיל. הוסיפו מתודות למחלקה זו כרצונכם ע"פ הצורך והדרישות בהמשך. מי שהולך ליצור מופעים של Maze יהיה הטיפוס MazeGenerator.

במשימה זו נתרגל את כתיבת ההיררכיה של המחלקות עבור אלגוריתם. את החלק של הבעיה נתרגל בחלק הבא של הפרויקט.

- הגדירו ממשק בשם IMazeGenerator שמגדיר:
 - מתודה בשם generate שמחזירה מופע של Maze. המתודה מקבלת שני פרמטרים, מס' שורות ומס' עמודות (כ- int).
 - מתודה בשם measureAlgorithmTimeMillis שמקבלת שני פרמטרים, מספר שורות ומספר עמודות (כ- int), ליצירת מבוך, ומחזירה long.
 - ממשו מחלקה אבסטרקטית כסוג של IMazeGenerator, קראו לה AMazeGenerator.
 - היא תשאיר את המתודה generate כאבסטרקטית. כל אלג' יממש זאת בעצמו.
 - לעומת זאת, פעילות מדידת הזמן זהה לכל האלגוריתמים, ולמעשה אינה תלויה באלגוריתם עצמו. לכן אותה דווקא כן נממש כאן (במקום לממש אותה כקוד כפול בכל אחת מהמחלקות הקונקרטיות)
 - המתודה measureAlgorithmTimeMillis תדגום את שעון המערכת ע"י System.currentTimeMillis(), תפעיל את generate עם הפרמטרים ליצירת מבוך שקיבלה, ותדגום את הזמן מיד לאחר מכן. הפרש הזמנים מתאר את הזמן שלקח להפעיל את generate. החזירו את זמן זה כ- long.
 - ממשו מחלקה בשם EmptyMazeGenerator (שתירש את המחלקה האבסטרקטית) שפשוט מייצרת מבוך ריק, חסר קירות.
 - ממשו מחלקה בשם SimpleMazeGenerator (שתירש את המחלקה האבסטרקטית) שפשוט מפזרת קירות בצורה אקראית באופן שמאפשר הרבה פתרונות אפשריים. קיימות דרכים רבות לעשות זאת.
 - למידה עצמית – עיקר התרגיל. היכנסו לעמוד הבא וויקיפדיה: https://en.wikipedia.org/wiki/Maze_generation_algorithm בחרו את אחד האלגוריתמים שאתם מתחברים אליו יותר. באמצעותו ממשו מחלקה בשם MyMazeGenerator, היורשת גם היא את המחלקה האבסטרקטית ומייצרת מבוך ע"פ הייצוג לעיל. דאגו שהאלגוריתם שלכם מייצר מבוכים מעניינים עם מבויים סתומים והתפצליות כפי הדוגמה:
 - כתבו את האלגוריתם בצורה יעילה כך שיוכל לייצר מבוך בגודל 1000*1000 בזמן סביר של עד דקה. חשבו על מבני הנתונים הנכונים להשתמש בהם כדי ליעל את האלגוריתם שלכם מבחינת סיבוכיות זמן ומקום.
 - שימו לב שאלגוריתמי החיפוש הם גנריים לחלוטין. הם אינם מודעים לבעיה אותה הם פותרים. האלגוריתמים מחפשים בעץ המצבים האפשריים, מהמצב ההתחלתי, במטרה למצוא את מצב היעד.
- ** ישנם אלגוריתמים המתייחסים לכל תא במבוך כמוקף ב- 4 קירות ובתהליך היצירה הם שוברים את הקירות באופן שמייצר מבוך. מבוכים הנוצרים נראים כך:**



**** תוכלו להבין את האלגוריתמים מסוג זה ולשנות אותם כך שיצרו את המבוך בפורמט המתבקש.**



בדיקות

הוסיפו לפרויקט שלכם Package חדש בשם test. ה-Package יכיל מספר מחלקות הניתנות להרצה (כוללות פונקציית main) וכל מחלקה תבדוק קטעי קוד אחרים. הוסיפו מחלקה בשם RunMazeGenerator המכילה את הקוד הבא:

```
package test;
import algorithms.mazeGenerators.*;
public class RunMazeGenerator {
    public static void main(String[] args) {
        testMazeGenerator(new EmptyMazeGenerator());
        testMazeGenerator(new SimpleMazeGenerator());
        testMazeGenerator(new MyMazeGenerator());
    }

    private static void testMazeGenerator(IMazeGenerator mazeGenerator) {
        // prints the time it takes the algorithm to run
        System.out.println(String.format("Maze generation time(ms): %s", mazeGenerator.measureAlgorithmTimeMillis(100/*rows*/,100/*columns*/)));
        // generate another maze
        Maze maze = mazeGenerator.generate(100/*rows*/, 100/*columns*/);

        // prints the maze
        maze.print();

        // get the maze entrance
        Position startPosition = maze.getStartPosition();

        // print the start position
        System.out.println(String.format("Start Position: %s", startPosition)); // format "{row,column}"

        // prints the maze exit position
        System.out.println(String.format("Goal Position: %s", maze.getGoalPosition()));
    }
}
```

שימו לב ש:

- הקוד נדרש לרוץ במלואו ללא שגיאות.
- המחלקה maze מכילה את השיטות הבאות:
 - getStartPosition – מחזיר את נקודת ההתחלה של המבוך (טיפוס מסוג Position).
 - getGoalPosition – מחזיר את נקודת הסיום של המבוך (טיפוס מסוג Position).
 - Print – מדפיסה את המבוך למסך. סמנו את נקודת הכניסה למבוך בתו S ואת נקודת היציאה בתו E.
- כחלק ממימוש האלגוריתם שמייצר מבוך, תצטרכו לקבוע למבוך מהי נקודת ההתחלה ומהי נקודת הסיום של המבוך.
- עליכם ליצור מחלקה בשם Position המייצגת מיקום בתוך המבוך. מקמו את המחלקה לצד המחלקה Maze (תחת אותו ה-Package). למחלקה יהיו שני Data Members שייצגו את השורה והעמודה. ההדפסה של Position בקריאה מתוך System.out.println צריכה להחזיר את המיקום בפורמט {row,column}. המחלקה תכיל את המתודות הבאות:
 - getRowIndex()
 - getColumnIndex()
- ה-main בוחן את שני האלגוריתמים, קוד ה-test לא היה צריך להשתנות!

משימה ב' – אלגוריתמי חיפוש

ניקוד החלק: 45 נקודות.

תחת החבילה algorithms צרו חבילה בשם search. בהתאם לתשתית שראינו בהרצאה:

1. צרו את:

a. המחלקות: ASearchingAlgorithm, AState, MazeState, Solution.

b. הממשקים: ISearchingAlgorithm, ISearchable.

2. ממשו את אלגוריתמי החיפוש הבאים:

a. חיפוש לרוחב Breadth First Search – קראו למחלקה BreadthFirstSearch.

b. חיפוש לעומק Depth First Search – קראו למחלקה DepthFirstSearch.

c. אלגוריתם Best First Search – קראו למחלקה BestFirstSearch.

3. כתבו את אלגוריתמי החיפוש כך שיהיו יעילים מבחינת סיבוכיות זמן ומקום. האלגוריתמים צריכים להתמודד עם מבוכים בגודל $1000 * 1000$ בזמן סביר של עד דקה. שימו לב שמבנה המבוך שלכם (תלוי באלגוריתם שיצר אותו) משפיע על כמות המצבים שאלגוריתם החיפוש יצטרך לפתח במהלך החיפוש.

4. צרו **Object Adapter** שמבצע אדפטציה ממבוך (מופע של Maze) לבעיית חיפוש

(ISearchable), קראו לו SearchableMaze.

a. במימוש המתודה getAllPossibleStates החזירו גם תנועות באלכסון (בנוסף לתנועה

אפשריות כמו שמאלה, ימינה, למעלה ולמטה).

b. תנועה אחת באלכסון מתא X לתא Y אפשרית רק אם ניתן להגיע מתא X ל-Y בשני

צעדים רגילים ללא אלכסון (תנועה בצורת 'r').

שימו לב ש Best First Search דומה מאד ל Breadth First Search פרט לעובדה שהראשון

משתמש בתור עדיפויות (Priority Queue). מומלץ שתור העדיפויות יהיה ממומש כערמה (Heap)

לשיפור הביצועים. שימו לב שאלגוריתם Breadth First Search מדרג מצבים לפי כמות הצעדים

מנקודת ההתחלה, כלומר צעד רגיל וצעד באלכסון נחשבים שניהם "צעד". לעומת זאת אלגוריתם

Best First Search מדרג מצבים לפי עלות ההגעה אליהם מנקודת ההתחלה, כאשר לצעד רגיל

ולצעד באלכסון עלות שונה. ניתן לומר ש Best הוא סוג של ספציפי יותר של Breadth ולכן על

המחלקה של Best לרשת את זו של Breadth ולדרוס את התור עם תור עדיפויות. מצד שני, ניתן

לומר שמדובר באותו האלגוריתם ובאותו המימוש, פשוט ל- Breadth First Search נזין שלכל

הקודקודים עדיפות שווה (צעד באלכסון שקול לצעד רגיל). שתי התשובות נכונות.

נשים לב שמספר הקודקודים שכל אלגוריתם ייפתח (מוציא מה open list) הוא שונה. ככל שמפתחים

פחות קודקודים כך האלגוריתם יותר יעיל. כדי להבין זאת ולחוש את האלגוריתמים השונים, **לפני**

המימוש בקוד, ענו כמה קודקודים ייפתח כל אלג' עבור הדוגמא הבאה:

נתון לנו מבוך שכל תנועה ישרה עולה 10 נקודות, ותנועה באלכסון עולה 15 נקודות (שימו לב שהיא

יותר חסכונית משתי תנועות בעלות של 20 המביאות לאותה הנקודה).

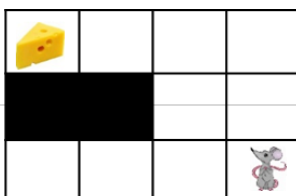
נגדיר "מצב" (State) כמיקום של העכבר במבוך (עמודה, שורה). במבוך העכבר נמצא ב(2,3).

נגדיר שבהנתן מצב, סדר פיתוח השכנים הוא עם כיוון השעון כשמתחילים מלמעלה. כלומר, עבור

המצב הנוכחי סדר פיתוח השכנים הוא 1. (1,3) 2. (1,4) 3. (2,4) וכך הלאה עד שנגיע ל (1,2).

כמובן שלא נרצה לפתח מצבים שנמצאים מחוץ למבוך או מייצגים קירות. עבור כל אלגוריתם

מצאו את מספר הקודקודים שהוא יפתח עד שימצא את המסלול הזול ביותר בדרך לגבינה.



בדיקות

תחת ה-Package לבדיקה שיצרתם (test) הוסיפו מחלקה בשם RunSearchOnMaze. צרו במחלקה מתודה Main ש:

a. יוצרת מבוך מורכב באמצעות MyMazeGenerator בגודל 30*30.

b. פותרת אותו באמצעות כל אחד משלושת מהאלגוריתמים.

i. BreadthFirstSearch

ii. DepthFirstSearch

iii. BestFirstSearch

c. עבור כל אלגוריתם:

i. מדפיסה למסך כמה מצבים פיתח. אם לא מורגש הבדל, הגדילו את המבוך.

ii. מדפיסה למסך את רצף הצעדים מנקודת ההתחלה לנקודת הסיום.

הריצו את הקוד הבא:

```
package test;

import algorithms.mazeGenerators.IMazeGenerator;
import algorithms.mazeGenerators.Maze;
import algorithms.mazeGenerators.MyMazeGenerator;
import algorithms.search.*;

import java.util.ArrayList;

public class RunSearchOnMaze {
    public static void main(String[] args) {
        IMazeGenerator mg = new MyMazeGenerator();
        Maze maze = mg.generate(30, 30);
        SearchableMaze searchableMaze = new SearchableMaze(maze);

        solveProblem(searchableMaze, new BreadthFirstSearch());
        solveProblem(searchableMaze, new DepthFirstSearch());
        solveProblem(searchableMaze, new BestFirstSearch());
    }

    private static void solveProblem(ISearchable domain, ISearchingAlgorithm
searcher) {
        //Solve a searching problem with a searcher
        Solution solution = searcher.solve(domain);
        System.out.println(String.format("%s' algorithm - nodes evaluated:
%s", searcher.getName(), searcher.getNumberOfNodesEvaluated()));
        //Printing Solution Path
        System.out.println("Solution path:");
        ArrayList<AState> solutionPath = solution.getSolutionPath();
        for (int i = 0; i < solutionPath.size(); i++) {
            System.out.println(String.format("%s.
%s", i, solutionPath.get(i)));
        }
    }
}
```

- מצופה שאלגוריתם Best First Search ימצא את המסלול הקצר ביותר בין נקודת ההתחלה לנקודת הסיום, מסלול הכולל אלכסונים במידה וניתן.

משימה ג' – 3D-Maze – משימת רשות

ניקוד החלק: 10 נקודות.

- בתוך ה Package של test צרו מחלקה חדשה שתקרא GeneralCheckingFunctions, שתעזור לבדוק להבין את הבחירות שלכם במהלך הפרויקט.
- שימו לב – המשימה הזאת היא משימת רשות בלבד שלא תשפיע על החלקים הבאים של העבודה.
- עליכם להוסיף למחלקה של GeneralCheckingFunctions את הפונקציה הבאה:

```
public static boolean check3DMaze(){  
    // if you have chosen to not do the Maze3D assignment, please change this value to  
    false.  
    // Note, that if this will be false, you will automatically get minus 10 in the score  
    of the first part of the project.  
    boolean weChoseToDoTheMaze3DAssignment = true;  
    return weChoseToDoTheMaze3DAssignment;  
}
```

- שימו לב, שמי שלא מעוניין לבצע את משימת המבוך התלת מימדי, צריך לשנות את הערך בפונקציה הזאת לfalse.
- נא לא לנסות לכתוב true ללא מימוש החלק הזה, דבר זה יגרור הורדת ניקוד נוסף.

כעת, לאחר שבניתם מבוך דו מימדי בהצלחה, וגם הצלחתם לפתור את המבוכים הדו מימדיים בעזרת אלגוריתמי החיפוש שלכם, כעת עליכם ליצור מבוך תלת מימדי ולפתור אותו גם כן. שימו לב, מכיוון שהשתמשתם באלגוריתמי חיפוש גנריים שפותרים כל בעיית חיפוש שמממשת את הממשק `ISearchable`, לא אמור להיות שינוי כלל באלגוריתמי החיפוש שלכם כשאתם פותרים את המבוך התלת מימדי, אלא רק באלגוריתם של יצירת המבוך והדרך שבה אתם מייצגים את המבוך התלת מימדי.

הגדרת הבעיה:

בעיית מבוך תלת מימדי דומה מאוד להגדרת המבוך הדו מימדי, רק שכעת הדמיות יכולה לנוע גם פנימה והחוצה, ולא רק למעלה, למטה, ימינה ושמאלה. נייצג את המבוך התלת מימדי באופן דומה לייצוג הדו מימדי, רק שכאן במקום מערך דו מימדי של `int`, נשתמש במערך **תלת** מימדי. דוגמה (נקודת ההתחלה מסומנת באדום ונקודת הסיום בירוק. מסלול הפתרון מסומן בצהוב):

```
int[][][] maze = {  
    {  
        {0, 0, 1, 0, 1},  
        {0, 1, 1, 1, 0},  
        {0, 1, 1, 0, 0}  
    },  
    {  
        {1, 1, 1, 0, 1},  
        {1, 0, 0, 1, 0},  
        {0, 0, 1, 0, 1}  
    },  
    {  
        {1, 1, 1, 0, 1},  
        {1, 1, 0, 0, 0},  
        {1, 1, 1, 0, 1}  
    }  
};
```

עליכם לחשוב איך לממש את המבוך התלת מימדי מבחינת היררכיית המחלקות שבניתם לפני כן.

צרו Package חדש תחת ה Package של `algorithms`, השם של ה package החדש יהיה: `maze3D`. בתוך ה package החדש עליכם לייצר את המחלקות הבאות:

1. interface חדש שיקרא `IMazeGenerator3D` שיגדיר את השיטה:

```
Maze3D generate(int depth, int row, int column);
```

וגם את השיטה:

```
long measureAlgorithmTimeMillis(int depth, int row, int column);
```

2. מחלקה אבסטרקטית חדשה שתקרא AMaze3DGenerator שתתממש את ה interface שיצרתם,

ותממש את השיטה `measureAlgorithmTimeMillis`.

3. ממשו מחלקה שתקרא MyMaze3DGenerator, שתרחיב את המחלקה האבסטרקטית שיצרתם. על המחלקה הזאת לממש את השיטה `generate`.

- עליכם לעשות שהמחלקה הזאת תממש מבוכים מעניינים, כלומר, אין צורך כעת לממש מבוכים ריקים או פשוטים כמו שהיה צריך במבוכים הדו מימדיים, אבל כן יש צורך להשתמש באלגוריתם מעניין שניתן למצוא בדף הויקיפדיה שקיבלתם בחלק של המבוכים הדו מימדיים, ולסדר שהוא יעבוד גם ליצירה של מבוכים תלת מימדיים – חשבו איך להתאים את האלגוריתם בצורה נכונה על מנת שיתאים גם למבוכים תלת מימדיים.

- שימו לב, שיש צורך גם ליצור מחלקות של Maze3D וגם Position3D. על המחלקה Maze3D לאפשר החזרה של מפת המבוך וגם של נקודות ההתחלה והסיום, על ידי השיטות:

- `public int[][][] getMap()`
- `public Position3D getStartPosition()`
- `public Position3D getGoalPosition()`

- על המחלקה Position3D לאפשר החזרה של כל הקואורדינטות של המיקום, על ידי השיטות:

- `public int getDepthIndex()`
- `public int getRowIndex()`
- `public int getColumnIndex()`

4. לאחר שהצלחתם ליצור מבוכים תלת מימדיים, עליכם גם לאפשר פתירה של המבוכים הללו. שימו לב, שבעקבות הגנריות של ה solvers שיצרתם, אתם אמורים להצליח לפתור כל בעיה שמממשת את הממשק ISearchable. לכן, עליכם ליצור כעת שתי מחלקות שיאפשרו לכם לפתור מבוכים תלת מימדיים. המחלקות הן:

- SearchableMaze3D, שיממש את הממשק ISearchable.
- שימו לב כי כעת אתם לא צריכים לתמוך בהליכה באלכסון כמו שהיה צריך במבוך הדו מימדי.
- Maze3DState שתרחיב את המחלקה AState.

כעת, עליכם לבדוק את עצמכם. הוסיפו שתי מחלקות בדיקות תחת ה Package של test. המחלקות יקראו:

- RunMaze3DGenerator שתבדוק יצירה של מבוכים תלת מימדיים – וודאו שהיצירה של המבוכים לא לוקחת יותר מידי זמן, למשל יצירה של מבוך תלת מימדי עם המימדים: `depth=rows=columns=500`, אמורה לקחת זמן סביר של עד דקה. שימו לב שעליכם לוודא גם שהמבוכים שאתם יוצרים הם פתירים, מומלץ ליצור מבוכים קטנים ולראות אם אתם מצליחים לפתור אותם, לפני שאתם משתמשים גם ב Solvers.
- RunSearchOnMaze3D שתפתור מבוכים תלת מימדיים על ידי ה solvers שיצרתם לפני כן.

משימה ד' – Unit Testing (למידה עצמית)

ניקוד החלק: 5 נקודות.

עוד לפני שנתחיל לכתוב את ה GUI נכיר עוד כלי שמאפשר לנו לבדוק את הקוד בפרויקט – Unit Testing.

תפקידינו כמפתחים הוא גם לבדוק את המחלקות שהוא אחראי להן. הוא מתחייב שכל מחלקה שהוא מעלה ל repository היא בדוקה ונמצאה אמינה. מאוחר יותר ה QA בודק האם החלקים השונים של הפרויקט מדברים זה עם זה כמו שצריך ואין בעיות שנוצרות ביניהם.

אחד הכלים המוצלחים נקרא JUnit. הרעיון הוא שלכל מחלקה חשובה שכתבנו תהיה לה גם מחלקת JUnit test שבדוקת אותה. כך, לאחר שביצענו שינויים בקוד, נריץ תחילה את ריצת הבדיקה, ואם כל הבדיקות "עברו" אז נוכל להריץ את הפרויקט ולהעלות אותו ל repository. במידה ולא עברו, נוכל לפי הבדיקה שכשלה לבודד את התקלה שגרמנו בעקבות השינוי. כך נחסך זמן פיתוח רב.

להלן הסבר על התקנת JUnit5 ב IntelliJ, וסידור תיקיית Test חדשה בפרויקט. שימו לב, עליכם לקרוא לתיקיית ה Test החדשה בשם "JUnit" ולהגדיר אותה כ "Tests" בתוך ה Modules section שנמצאת ב Project Structure (Ctrl+Alt+Shift+S):

<https://www.jetbrains.com/help/idea/testing.html>

להלן דוגמה לעבודה עם JUnit ב-IntelliJ:

<https://www.youtube.com/watch?v=Bld3644blAo>

ישנם הגורסים שאת קוד הבדיקות יש לכתוב עוד לפני שכותבים את המחלקה הנבדקת עצמה. כך, הבדיקה תיעשה ללא ההשפעה של הלך המחשבה שהוביל לכתובת המחלקה, ועלול להיות מוטעה.

עליכם ליצור מחלקת בדיקה לאלגוריתם Best First Search, קראו למחלקה BestFirstSearchTest. מקמו את המחלקה בתיקיית בדיקה חדשה בשם JUnit. השתמשו ב JUnit 5.

- שימו לב, אם תעשו כפי שמוסבר בסרטון הדוגמה, מחלקת הבדיקה אמורה להיווצר לכם באופן אוטומטי ב Package הבדיקות החדש שיצרתם (JUnit).

מה בודקים? לא את נכונות האלגוריתם, בשביל זה יש מדעני מחשב. עליכם לבדוק את המימוש של האלגוריתם. כיצד הוא מתנהג עבור פרמטרים שגויים? Null? תבדקו מקרי קצה.

הכל עובד?

מצוין!

משימה ה' – עבודה עם מנהל גרסאות (למידה עצמית)

ניקוד החלק: 5 נקודות.

מעתה אתם עובדים בזוגות. אנו מדמים את המציאות בה אנו מתכנתים כחלק מצוות תכנות. אחד האתגרים הוא ניהול העבודה, ובפרט ניהול הקוד. לא נרצה שתדרסו את הקוד של אחד ע"י השני, או שתלכו לאיבוד בין אינסוף גרסאות ששלחתם במייל... כמו כן, נרצה לשמור גרסאות קודמות כדי שנוכל לחזור לגרסה עובדת במקרה של תקלות או כדי לתמוך במשתמשים בעלי גרסאות קודמות של המוצר שלנו.

לשם צוותי פיתוח משתמשים במנהל גרסאות.

הרעיון הוא פשוט: בתחילת יום עבודה מורידים ממאגר הקוד שלנו (היושב על שרת כלשהו) את הגרסה האחרונה. עושים את השינויים שלנו על עותק מקומי. לאחר שאנו בטוחים שהשינוי עובד כראוי אנו מעלים אותו חזרה לשרת ומעדכנים את כולם בקוד שלנו. עליכם להתחיל לעבוד כמו המקצוענים.

בפרויקט זה עליכם לעבוד מול GIT. לימדו עצמאית כיצד להגדיר מאגר. יש המון הדרכות וסרטוני

הדרכה בנושא ברשת המדגימים את הנושא. כעת צרו פרויקט hello world פשוט ותתנהלו מולו עם שינויים בקוד עד שתבינו כיצד לנהוג כשותפים לאותו מאגר קוד. לאחר שהבנתם כיצד לעבוד יחד תוכלו להתחיל את העבודה על המטלה בפרויקט חדש. ניתן להיעזר למשל בקישור לסרטון שנמצא במודל תחת החלק של חומר עזר, בחלק של Version Control.

1. על כל אחד מהסטודנטים ליצור חשבון Github משלו.
2. אחד הסטודנטים צריך לפתוח פרויקט פרטי (Private repository) ב Github, ולשתף אותו לחבר הצוות השני. עליכם לשתף גם את בודק התרגילים בפרויקט שלכם, על מנת שיוכל לעבור עליו בזמן הבדיקה. **שם המשתמש של בודק התרגילים של הקורס ב Github הוא: nitay16**
3. הוסיפו למחלקה GeneralCheckingFunctions שיצרתם במשימה ג את הפונקציה הבאה:

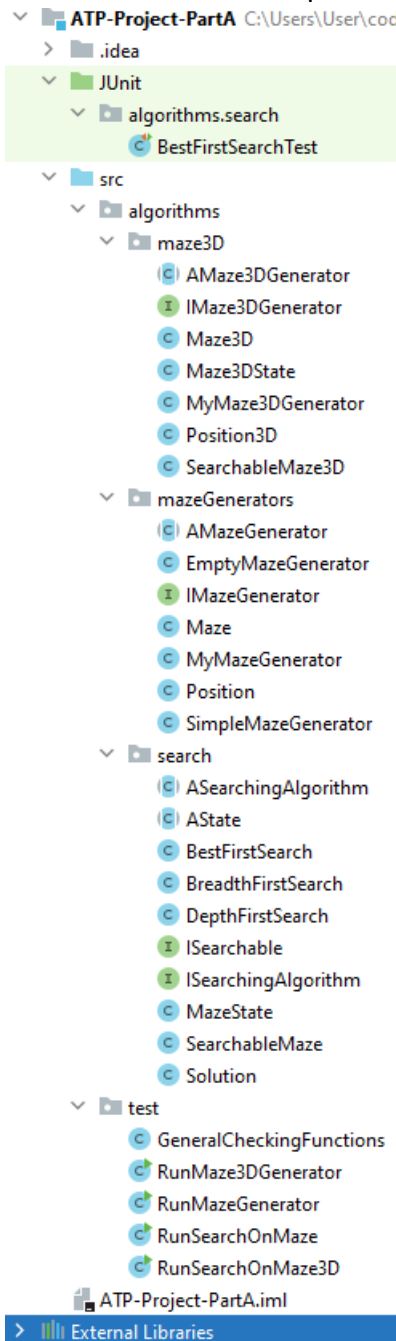
(שימו לב – יש לשנות את הלינק שבפונקציה להיות הלינק לפרויקט שלכם ב github):

```
public static String getGithubLink(){  
    //change the <username> in the link to the username of the student who  
    created the github project:  
    String githubLink = "https://github.com/<username>/ATP-Project";  
    return githubLink;  
}
```

4. במהלך כל הפרויקט עליכם להשתמש ב github בצורה נכונה, כלומר, לאחר שסיימתם חלק משמעותי בקוד, עליכם לעשות לו commit. שימו לב, העבודה מול Github **תיבדק ותקבלו ציון על כך בכל חלקי הפרויקט. הציון על העבודה מול Github יהיה 5 נקודות מכל חלק בפרויקט.**

דגשים להגשה

בדקו שהפרויקט שלכם מכיל את החבילות, המחלקות והממשקים בפורמט הבא במדויק:



**** שימו לב שיש חשיבות לאותיות גדולות וקטנות בטקסט (Case Sensitive).**

**** וודאו שקוד הבדיקה שניתן לכם רץ אצלכם ועובד עם הקוד שלכם כמו שהוא בלי שערכתם בו שינויים.**

**** שימו לב, אם בחרתם לא לבצע את משימה ג', השוני היחיד שאמור להיות לכם בפרויקט הוא שלא יהיה לכם את ה Package של Maze3D, ואת מחלקות הבדיקה של: RunMaze3DGenerator, RunSearchOnMaze3D.**

בהצלחה!

חלק ב': עבודה עם Streams ו-Threads

הקדמה

בהמשך הפרויקט אנו ניצור זוג שרתים שנותנים שרות לשלל לקוחות. תפקיד השרת הראשון לייצר מבוכים לפי דרישה. תפקיד השרת השני לפתור מבוכים. כאשר לקוח מתחבר לשרת שיוצר מבוכים הוא ישלח לו את הפרמטרים ליצירת המבוך ויקבל חזרה אובייקט מבוך. כאשר לקוח מתחבר לשרת שפותר מבוכים הוא ישלח לו מבוך קיים ויקבל חזרה מהשרת פתרון למבוך. כדי לקצר את זמן התקשורת יהיה עלינו לדחוס את המידע שעובר ביניהם טרם השליחה. הצד המקבל יפתח את הדחיסה וייהנה מהמידע. עוד דבר שנעשה בצד השרת זה לשמור פתרונות שכבר חישבנו, כך שאם נתבקש לפתור בעיה שכבר פתרו נשלף את הפתרון מהקובץ במקום לחשב אותו מחדש. תהליך זה מכונה caching.

לשם כך, בחלק זה של המטלה אנו נתרגל עבודה עם קבצים, נממש אלג' דחיסה פשוט, ונכיר כמה מחלקות מוכנות. כדי שנוכל לעבוד עם הדחיסה שלנו גם מעל ערוץ תקשורת וגם מעל קבצים נצטרך לכתוב את הקוד שלנו כחלק מה decorator pattern של מחלקות ה streams ב-Java. זה ייתן לנו גמישות מרבית, שכן, המקור ממנו מגיע המידע לא קריטי לנו.

תוכלו להמשיך לפתח את חלק זה (חלק ב') על גבי הפרויקט שכבר שהגשנתם (חלק א'). לפני שתמשיכו גבו את חלק א' שכבר הגשנתם. שנו את שם הפרויקט שלכם ל- ATP-Project-PartB.

משימה א' – דחיסה של Maze ו-Decorator design pattern

כמה מידע באמת מחזיק מופע של Maze?

הוא מחזיק את נק' הכניסה והיציאה מהמבוך, וכמובן את הגדרת המבוך. הגדרה זו די בזבזנית. כך שאם Maze תהיה serializable ונשלח מופע שלה בתקשורת או נשמור אותו בקובץ, בזבזנו המון מקום מיותר.

איך ניתן לכווץ את המידע?

דבר ראשון אנו משתמשים ב int (גודל של 4 בתים) כדי לייצג את הערכים 0 או 1, חבל. להשתמש ב byte בודד כבר יחסוך לנו פי 4 בתים.

כעת, דמיינו שאתם פורסים את המערך הדו-מימדי שמייצג את המבוך שלנו למערך חד-מימדי ארוך. יש בו המון רצפים שחוזרים על עצמם, לדוגמא:

1,1,1,1,1,1,0,0,0,0,0,0,0,0,1,1,1,1,1,...

במקום לשמור את הרצף עצמו, נוכל לשמור את מספר ההופעות ברצף של כל ספרה. עלינו להגדיר עם איזו ספרה אנו מתחילים, למשל 0, ואז נרשום את מספר ההופעות ברצף של 0, המספר הבא יהיה מספר ההופעות ברצף של 1, ואחריו שוב עבור 0 וחוזר חלילה. עבור הדוגמא לעיל נקבל: 0,7,10,6. כך נדע ש 0 לא מופיע, 1 מופיע 7 פעמים ברצף, ואז 0 מופיע 10 פעמים ברצף, ואז 1 6 פעמים ברצף וכך הלאה...

נצטרך משמעותית פחות בתים כדי לייצג את אותו המידע בדיוק.

נשים לב שאם אנו משתמשים בבתים אז המקסימום שנוכל לכתוב הוא 255 (unsigned). כך שאם למשל "1" הופיע 300 פעם ברצף, נצטרך לכתוב 255,0,45 כלומר 1 הופיע 255 פעמים ברצף, 0 לא הופיע, ואז 1 הופיע 45 פעם ברצף. אם ניצור מחלקה שדוחסת את המידע של המבוך, שומרת את ממדיו הדו-מימדיים, וכן שומרת את מיקומי הכניסה והיציאה מהמבוך, אז נוכל באמצעותה לשמור \ להעביר את המבוך בכמות פחותה של בתים באופן משמעותי, ולאחר מכן לשחזר בדיוק את אותו המבוך.

ניתן כמובן לחשוב על דרכים יעילות יותר לדחוס את המידע ואתם נדרשים לממש דחיסה בצורה החסכונית ביותר שתוכלו.

איך נממש זאת נכון מבחינת design? נשתלב ב decorator pattern של ה streams ב java.

בספריית הקוד שלנו צרו חבילה בשם IO (מקבילה לחבילה algorithms) בתוכה, צרו מחלקה בשם `MyCompressorOutputStream`. מחלקה זו תירש את `OutputStream` ותקבל בבנאי שלה `OutputStream`. זה כמובן יחייב אתכם לממש את `void write (int b)`. שימו לב שניתן לדרוס גם את המתודה `void write(byte[] b)`.

כעת, צרו data member בשם `out` מהסוג `OutputStream` ותחלו אותו בבנאי עם מופע של `OutputStream` שקיבלתם. את `write` תממשו בהמשך באמצעותו.

נניח שמחלקה זו קיבלה מערך של בתים לכתוב, היא תפעיל את `write` עבור כל אחד מבתיים אלה. כל שעליכם לעשות הוא לבדוק האם `b` הוא בית חדש או שהוא חזר על עצמו מההפעלה הקודמת של `write`. כל עוד זו חזרה אז נצבור את הפעמים. ברגע שנקבל בית חדש, נשתמש ב `out` כדי לרשום גם את את מספר ההופעות שלו, ונתחיל את הצבירה מחדש עבור הבית החדש שקיבלנו זה עתה. באופן דומה תוכלו לממש את הכיוון ההפוך במחלקה `MyDecompressorInputStream`, שתממש את `InputStream` באמצעות מופע של `InputStream` שתקבל בבנאי. תקראו לו `in`. באמצעות `in` נקרא מידע מכוון ממקור המידע שלנו. "ננפח" את המידע בהתאם לשיטת הכיוון לעיל ונזין את מי שקורא מידע ממחלקה זו במידע המנופח.

.write
התו וגם

כעת, במחלקה `Maze` נוסיף שני דברים:

1. את המתודה `toArray` שתחזיר `byte[]` המייצג את כל המידע (הלא מכווץ) של המבוך: ממדי המבוך, תוכן המבוך, מיקום כניסה, מיקום יציאה. החליטו בעצמכם על הפורמט שבאמצעותו תייצגו את המבוך כ- `byte[]`. נסו להיות חסכוניים ככל האפשר.
2. בנאי שמקבל מערך של `byte` לא מכווץ (לפי הפורמט שאתם מחזירים מהסעיף הקודם) ובונה באמצעותו את `Maze`.

- שימו לב – עליכם לממש שני מימושים שונים של כיוון המבוך.
- מימוש של מחלקות `SimpleCompressorOutputStream`, `SimpleDecompressorInputStream`, שיממשו כיוון לפי השיטה שנתונה בעבודה (ספירה של כמות רצפי האחדות והאפסים במבוך). המימוש של המחלקות הללו יהיה באותו design כמו המחלקות `MyCompressorOutputStream`, `MyDecompressorInputStream`, הדבר היחיד ששונה זה **שיטת** הכיוון של המבוכים. הניקוד על הכיוון הזה יהיה 25 נקודות מסך הנקודות של חלק ב של הפרויקט.
- מימוש של `MyCompressorOutputStream`, `MyDecompressorInputStream`, שיממשו כיוון בשיטה לבחירתכם, הניקוד על הכיוון הזה יהיה יחסי לשאר הסטודנטים (כלומר זוג הסטודנטים עם הכיוון היעיל ביותר יקבל את כל הנקודות על הכיוון הזה, ושאר הסטודנטים יקבלו ניקוד בהתאם ליחס בין היעילות של הכיוון שלהם לבין היעילות של הכיוון היעיל ביותר). הניקוד היחסי על החלק הזה יהיה בגובה 15 נקודות מסך הנקודות של חלק ב של הפרויקט.

תחת ה-Package לבדיקה שיצרתם (test) הוסיפו מחלקה בשם RunCompressDecompressMaze.

צרו במחלקה מתודה Main:

```
package test;

import IO.MyCompressorOutputStream;
import IO.MyDecompressorInputStream;
import algorithms.mazeGenerators.AMazeGenerator;
import algorithms.mazeGenerators.Maze;
import algorithms.mazeGenerators.MyMazeGenerator;

import java.io.*;

public class RunCompressDecompressMaze {
    public static void main(String[] args) {
        String mazeFileName = "savedMaze.maze";
        AMazeGenerator mazeGenerator = new MyMazeGenerator();
        Maze maze = mazeGenerator.generate(100, 100); //Generate new maze

        try {
            // save maze to a file
            OutputStream out = new MyCompressorOutputStream(new
FileOutputStream(mazeFileName));
            out.write(maze.toByteArray())
            ; out.flush();
            out.close();
        } catch (IOException e) {
            e.printStackTrace();
        }

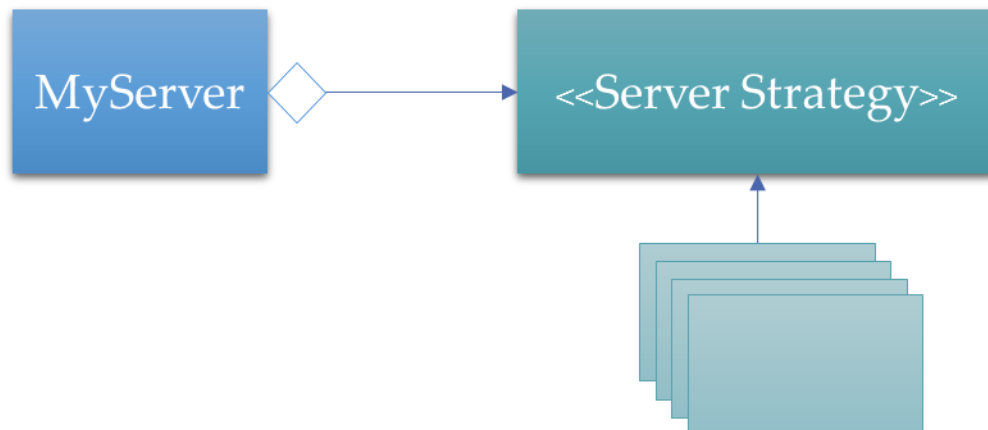
        byte savedMazeBytes[] = new byte[0];
        try {
            //read maze from file
            InputStream in = new MyDecompressorInputStream(new
FileInputStream(mazeFileName));
            savedMazeBytes = new byte[maze.toByteArray().length];
            in.read(savedMazeBytes);
            in.close();
        } catch (IOException e) {
            e.printStackTrace();
        }

        Maze loadedMaze = new Maze(savedMazeBytes);
        boolean areMazesEquals =
Arrays.equals(loadedMaze.toByteArray(), maze.toByteArray());
        System.out.println(String.format("Mazes equal: %s", areMazesEquals));
        //maze should be equal to loadedMaze
    }
}
```

בצעו את הניסוי הבא על מופע קיים של Maze, מה גודל המבוך בבתים? מה גודל המבוך בייצוג החסכוני? מה גודל הקובץ שנשמר בבתים? האם הקריאה מהקובץ הניבה מבוך זהה בדיוק?

משימה ב' - שרת-לקוח ו-Threads

בתרגול ראינו דוגמת שרת-לקוח המטפלת בלקוח אחד. העיצוב אפשר לנו ליצור שרת גנרי לשימוש חוזר בכל פרויקט באמצעות ה-Strategy Pattern.



בהרצאה ראינו דוגמת שרת-לקוח המטפלת בלקוחות במקביל, אך גם דיברנו על מס' מקרי קצה שניתן לפתור באמצעות thread pool. בתרגול, ראינו כיצד להשתמש ב thread pool מוכן מהספרייה java.util.concurrent.

צרו חבילה חדשה בשם Server (לצד IO) ובתוכה צרו את המחלקה Server שראיתם בתרגול יחד עם ה-ServerStrategy (שם חדש במקום ClientHandler) הבאים:

1. ServerStrategyGenerateMaze – השרת מקבל מהלקוח מערך int[] בגודל 2 בלבד כאשר התא הראשון מחזיק את מספר השורות במבוך, התא השני את מספר העמודות. השרת מייצר מבוך ע"פ הפרמטרים, דוחס אותו בעזרת MyCompressorOutputStream ושולח חזרה ללקוח byte[] המייצג את המבוך שנוצר.
 - a. הלקוח יקבל מהשרת את המערך, יפענח אותו, ויוכל בעזרתו מופע של מבוך תואם.
2. ServerStrategySolveSearchProblem – השרת מקבל מהלקוח אובייקט Maze המייצג מבוך. פותר אותו ומחזיר ללקוח אובייקט Solution המחזיק את הפתרון של המבוך.

משימות:

1. טפלו בקוד השרת הגנרי כך שיתמוך במספר לקוחות במקביל ע"י שימוש ב-thread pool, ויטפל במקרה הקצה של היציאה המסודרת. השרת והלקוח אינם מתחזקים קשר רציף אלא רק סשן של שאלה-תשובה. הלקוח שולח בקשה, השרת משיב בתשובה ואז הקשר נסגר. עבור בקשה חדשה יש לפתוח את התקשורת מחדש.
2. צרו שרת היוצר מבוכים ע"פ הפרוטוקול לעי"ל.
3. צרו שרת הפותר בעיות חיפוש ע"פ הפרוטוקול לעי"ל.
 - a. השרת שומר את הפתרון למבוכים שהוא מקבל בדיסק, כל פתרון נשמר בקובץ נפרד. את המבוכים שהשרת פותר תוכלו לשמור לתיקיה זמנית. על מנת לקבל את התיקיה הזמנית במערכת שלכם השתמשו ב:

```
String tempDirectoryPath = System.getProperty("java.io.tmpdir");
```

- b. אם השרת נדרש לפתור בעיה שהוא כבר פתר בעבר, הוא ישלף את הפתרון מהקובץ ויחזיר אותו ללקוח מבלי לפתור את הבעיה שוב.

צרו חבילה חדשה בשם Client (לצד IO) ובתוכה צרו את המחלקה Client שראינו בתרגול ואת הממשק IClientStrategy המגדיר את המתודה:

```
void clientStrategy(InputStream inFromServer, OutputStream outToServer);
```

השימוש במחלקה Client ובממשק clientStrategy הוא עבור בדיקת השרתים בלבד (מופיע בהמשך). אין חובה להשתמש במחלקה ו/או בממשק כאשר אתם נדרשים לפנות לשרת (שימו לב שהמתודה ClientStrategy של המחלקה Client אינה מחזירה ערך).

בדיקות

תחת ה-Package לבדיקה שיצרתם (test) הוסיפו מחלקה בשם RunCommunicateWithServers. צרו במחלקה מתודה Main:

```
public class RunCommunicateWithServers {
    public static void main(String[] args) {
        //Initializing servers
        Server mazeGeneratingServer = new Server(5400, 1000, new
        ServerStrategyGenerateMaze());
        Server solveSearchProblemServer = new Server(5401, 1000, new
        ServerStrategySolveSearchProblem());
        //Server stringReverserServer = new Server(5402, 1000, new
        ServerStrategyStringReverser());

        //Starting servers
        solveSearchProblemServer.start();
        ; mazeGeneratingServer.start();
        //stringReverserServer.start();

        //Communicating with servers
        CommunicateWithServer_MazeGenerating();
        CommunicateWithServer_SolveSearchProblem();
        //CommunicateWithServer_StringReverser();

        //Stopping all servers
        mazeGeneratingServer.stop();
        solveSearchProblemServer.stop();
        //stringReverserServer.stop();
    }

    private static void CommunicateWithServer_MazeGenerating() {
        try {
            Client client = new Client(InetAddress.getLocalHost(), 5400, new
            IClientStrategy() {
                @Override
                public void clientStrategy(InputStream inFromServer,
                OutputStream outToServer) {
                    try {
                        ObjectOutputStream toServer = new
                        ObjectOutputStream(outToServer);
                        ObjectInputStream fromServer = new
                        ObjectInputStream(inFromServer);
                        toServer.flush();
                        int[] mazeDimensions = new int[]{50, 50};
                        toServer.writeObject(mazeDimensions); //send maze
                        dimensions to server
                        toServer.flush();
                        byte[] compressedMaze = (byte[])
                        fromServer.readObject(); //read generated maze (compressed with
                        MyCompressor) from server
                        InputStream is = new MyDecompressorInputStream(new
                        ByteArrayInputStream(compressedMaze));
                        byte[] decompressedMaze = new byte[1000 /*CHANGE
                        SIZE ACCORDING TO YOU MAZE SIZE*/]; //allocating byte[] for the decompressed
                        maze -
                        is.read(decompressedMaze); //Fill decompressedMaze
```


with bytes

```
        Maze maze = new Maze(decompressedMaze);
        maze.print();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

});
client.communicateWithServer();
} catch (UnknownHostException e) {
    e.printStackTrace();
}
}

private static void CommunicateWithServer_SolveSearchProblem() {
    try {
        Client client = new Client(InetAddress.getLocalHost(), 5401, new
IClientStrategy() {
            @Override
            public void clientStrategy(InputStream inFromServer,
OutputStream outToServer) {
                try {
                    ObjectOutputStream toServer = new
ObjectOutputStream(outToServer);
                    ObjectInputStream fromServer = new
ObjectInputStream(inFromServer);
                    toServer.flush();
                    MyMazeGenerator mg = new MyMazeGenerator();
                    Maze maze = mg.generate(50, 50);
                    maze.print();
                    toServer.writeObject(maze); //send maze to server
                    toServer.flush();
                    Solution mazeSolution = (Solution)
fromServer.readObject(); //read generated maze (compressed with
MyCompressor) from server

                    //Print Maze Solution retrieved from the server
                    System.out.println(String.format("Solution steps:
%s", mazeSolution));
                    ArrayList<AState> mazeSolutionSteps =
mazeSolution.getSolutionPath();
                    for (int i = 0; i < mazeSolutionSteps.size(); i++) {
                        System.out.println(String.format("%s. %s", i,
mazeSolutionSteps.get(i).toString()));
                    }
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
        client.communicateWithServer();
    } catch (UnknownHostException e) {
        e.printStackTrace();
    }
}

private static void CommunicateWithServer_StringReverser() {
    try {
        Client client = new Client(InetAddress.getLocalHost(), 5402, new
IClientStrategy() {
            @Override
            public void clientStrategy(InputStream inFromServer,
OutputStream outToServer) {
                try {
                    BufferedReader fromServer = new BufferedReader(new
InputStreamReader(inFromServer));
                    PrintWriter toServer = new PrintWriter(outToServer);
```

```

        String message = "Client Message";
        String serverResponse;
        toServer.write(message + "\n");
        toServer.flush();
        serverResponse = fromServer.readLine();
        System.out.println(String.format("Server response:
%s", serverResponse));

        toServer.flush();
        fromServer.close();
        toServer.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

});
client.communicateWithServer();
} catch (UnknownHostException e) {
    e.printStackTrace();
}
}
}

```

משימה ג' – קובץ הגדרות (לימוד עצמי)

ניקוד החלק: 5 נקודות.

כמה ת'רדים צריך לאפשר ב Thread Pool? באיזה אלגוריתם לפתור את המבוכים? ובאיזה ליצור אותם?

כל אלו הן הגדרות. אסור לנו לקבוע אותן באופן שרירותי בקוד. המשתמש אולי ירצה לשנות אותן. גם השתמשנו במשתנים הנמצאים בתוך הקוד עבור ההגדרות האלו, אז כדי לשנות אותן נצטרך לשנות את קוד המקור שלנו ולבנות (לקמפל) את הפרויקט מחדש! רעיון גרוע מאד...

בתוך ה-Package שבו יצרתם את השרתים, צרו מחלקת סינגלטון בשם Configurations המאפשרת השמה ושליפה של כל ההגדרות שנראה לכם שצריך בתוכנית. לקובץ ההגדרות קראו config.properties, מקמו את הקובץ בתוך הפרויקט בתיקיית resources (לצד תיקיית src)

דאגו להגדיר את תיקיית ה-resources כתיקיית משאבים בהגדרות הפרויקט (open module settings).

השרתים שתצרו ישלפו את ההגדרות מתוך המחלקת ההגדרות ויפעלו בהתאם. וודאו שההגדרות הבאות מופיעות לכם בקובץ ההגדרות:

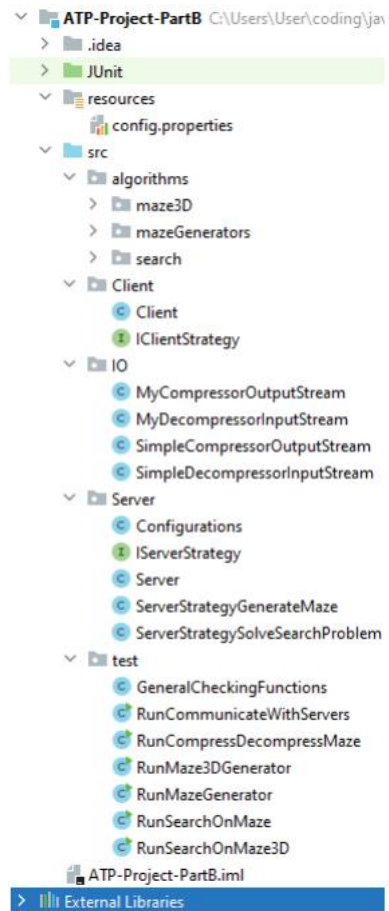
- **threadPoolSize** – הגדרה שקובעת את כמות הת'רדים שצריך לאפשר ב Thread Pool.
- **mazeGeneratingAlgorithm** – הגדרה שקובעת את שם אלגוריתם יצירת המבוכים שאתם רוצים להשתמש בו.
- **mazeSearchingAlgorithm** – הגדרה שקובעת את שם אלגוריתם פתירת המבוכים שאתם רוצים להשתמש בו.

להסבר על קובץ קונפיגורציה ב-Java:

<http://www.mkylong.com/java/java-properties-file-examples/>

דגשים להגשה

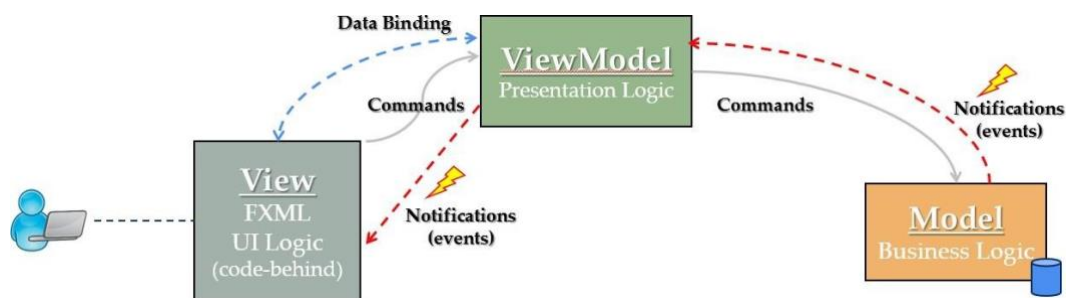
בדקו שהקוד שלכם מאורגן באופן הבא:



בהצלחה!

חלק ג': אפליקצית Desktop בארכיטקטורת MVVM, תכנות מונחה אירועים ו-GUI

משימה א' – ארכיטקטורת MVVM



- פתחו פרויקט חדש מסוג JavaFX ובתוכו צרו את ה-Packages הבאים המהווים שכבות קוד

שונות:

- View
- ViewModel
- Model

- בשכבת ה-View:

- מקמו את קובץ ה-fxml הראשי של הפרויקט ושנו את שמו ל-MyView.
- מקמו את ה-controller של קובץ ה-fxml ושנו את שמו ל-MyViewController.
- צרו ממשק בשם IView והגדירו את המתודות הרלוונטיות לשכבה.
- דאגו ש-MyViewController יממש את IView.

- בשכבת ה-Model:

- צרו את המחלקה MyModel יחד עם הממשק IModel שאותו הוא יממש.
- הגדירו בעצמכם מהן המתודות של IModel.

- בשכבת ה-ViewModel:

- צרו מחלקה בשם MyViewModel.

- צרפו לפרויקט את קבצי ה-JAR מחלקי הפרויקט הקודמים. שם הקובץ צריך להיות "ATPProjectJAR".

זה יאפשר לכם לעשות שימוש במחלקות שכתבתם באבני הדרך הקודמים (Maze, MazeGenerator, Server, etc.)

- בחרו בעצמכם אילו תכונות ה-MyView נרצה לכרוך (Bind) ל-MyViewController.

שימו לב:

- בשכבת ה-View בלבד יהיו כל החלקים הקשורים בתצוגה:

- קובץ או קבצי ה-fxml יחד עם ה-controllers שלהם.
- קבצי עיצוב CSS.
- פקדים שיצרתם בעצמכם.

- שכבת ה-Model בלבד אחראית על:

- התקשורת לשרתים שיוצרים ופותרים מבוכים.
- שימוש באלגוריתמים.
- שמירת המבוך שכרגע המשתמש משחק בו.
- שמירת מיקום הדמות במבוך הנוכחי.

- שכבת ה-ViewModel אחראית על החיבור בין שכבת ה-View לשכבת ה-Model.

משימה ב' – תכנות מונחה אירועים ו GUI

יצירת רכיב גרפי – תזכורת

א. כאשר אנו כותבים GUI כמו כל דבר אחר עלינו להקפיד על חלוקה נכונה למחלקות. לא הכל נכתב במחלקה אחת או ב-main אחד חלילה.

למשל כשאנחנו רוצים לממש לוח משחק או מבוך, מדובר באוסף של controls ו listeners שלא נרצה שיתערבבו יחד עם אלו שמגדירים את תוכן החלון (כפתורים תפריטים וכו'). כי אם נרצה להסיר את הרכיב הזה מהפרויקט או לחילופין להוסיף אותו לפרויקט אחר, תהיה לפנינו מלאכת תפירה קפדנית ומציקה.

תארו לכם לדוג' שאבקש להציג (רק) את המבוך בפרויקט אחר, או להחליף את המבוך בפרויקט שלכם במשחק לוח אחר... אם משחק הלוח שלכם אינו יחידה עצמאית אלא אוסף של מחלקות ו listeners מעורבים יחדיו במחלקה גדולה אחרת יהיה לכם מאד קשה לבצע זאת.

במקום זאת, את כל אוסף הרכיבים השונים שמרכיבים את אותו לוח משחק נאגד בתוך מחלקה אחת משלו; הרי, בדומה למחלקת Button, מדובר באלמנט אחד שלם ונפרד מהחלון שמכיל אותו. נרצה שכל מתכנת יוכל להוסיף או להסיר את האלמנט הזה באותה הקלות שהוא מוסיף או מסיר כפתור.

ב. העובדה שהקוד של לוח המשחק נמצא במחלקה אחת אינה מספיקה לכך כדי שמתכנתים אחרים יוכלו להוסיף את הרכיב הזה בקלות בפרויקט שלהם ולהשתמש בו. לשם כך עלינו ליצור משהו שהם כבר מכירים ויודעים כיצד לנהוג בו. במילים אחרות, אנו צריכים ליצור פקד מותאם אישית (widget).

בשיעור המעבדה השני אודות JavaFX למדנו כיצד ליצור control משלנו. ניתן לעשות זאת בקלות ע"י ירושה של Canvas שהוא כבר סוג של control שמימש לא מעט דברים ומאד גמיש לעבודה.

ג. שימו לב שה control שאותו אתם כותבים יודע **מתי** קורים דברים אך לא בהכרח יודע **מה** לעשות בנידון.

תארו לכם מצב שמחלקת Button היתה גם מגדירה מה קורה כשלוחצים על הכפתור. לא הינו יכולים להשתמש בכפתור הזה בפרויקטים אחרים כי לא הינו יכולים להגדיר לו מה לעשות כשהכפתור נלחץ. מצד שני, רק הכפתור יודע מתי הוא נלחץ ואז הוא זה שצריך להפעיל משהו. לכן, כפי שראיתם בשיעור, נעשה שימוש ב strategy pattern כדי להגדיר לכפתור **מה** הוא צריך ליזום כשהוא נלחץ.

זה נעשה ע"י הזנה של אובייקט מסוג Listener כלשהו לכפתור, וכשהכפתור נלחץ הוא יפעיל את המתודה שאנחנו מימשנו. כך, **מתי** שהכפתור יודע שצריך להפעיל משהו, הוא מפעיל את **מה** שאנו הגדרנו לו.

באופן דומה, גם לוח המשחק שלכם כ control לא צריך להגדיר מה לעשות; הוא צריך לקבל זאת מבחוץ. וזאת כדי שיוכלו להשתמש בו גם בפרויקטים אחרים בהם **אותם האירועים גוררים פעולות אחרות**.

לדוגמא, במבוך נרצה לצייר איזושהי דמות. תהיה לנו פונקציית ציור הדמות, ונפעיל אותה בכל פעם שנצטרך לצייר את הדמות כשהגיע הזמן להזיז אותה. אם נממש את הפונקציה הזו במחלקת המבוך אז היא תהיה קבועה במחלקה זו. כלומר, בכל פרויקט תמיד מופיעה אותה הדמות. לעומת זאת, אם נקבל מבחוץ את פונקציית ציור הדמות אז כל פרויקט יוכל להגדיר בעצמו כיצד הדמות תיראה בדרך הרלוונטית לאותו הפרויקט. המבוך יודע מתי לצייר את הדמות, אך לא כיצד לצייר אותה, ולכן מקבל זאת מבחוץ.

דוגמא נוספת, נניח שהחלטתם שצריך להיות כפתור עזרה, והוא חלק בלתי נפרד מאובייקט לוח המשחק (זה לגיטימי). שמחים וטובי לב, בעת לחיצה על כפתור העזרה, ב listener של הכפתור לא עשיתם שום דבר מיוחד חוץ מלפנות למי שיועד מה לעשות בנידון, נניח איזושהי controller. לכאורה

זה מצוין, הרי לא מימשתם בתוך ה listener מה לעשות, פשוט פניתם למישהו אחר.

אבל המימוש הזה ספציפי לפרויקט שלכם וכך הופך את לוח המשחק לפחות נייד. מי אמר שבפרויקט אחר יש לכם את אותו הcontroller? הדרך הנכונה יותר היא לקבל את המופע של הlistener הזה מבחוץ, כך בכל פרויקט יוכלו להגדיר מה לעשות כשכפתור העזרה נלחץ. למשל לפתוח חלון נוסף או דפדפן אינטרנט, או פשוט לפנות לcontroller שיעביר את הבקשה הלאה.

לסיכום, כשאתם מייצרים אלמנט גרפי, הקפידו על מימושו במחלקה משלו, מסוג widget, שמקבל מבחוץ את כל הפעולות שעשויות להיות שונות בפרויקטים שונים. כך, כולם יוכלו בקלות להסיר או להוסיף ואף להשתמש באלמנט הזה כרצונם בפרויקטים שלהם.

ד. הכנסת ה GUI שלכם לא צריכה לפגוע בשאר הקוד. שכבות ה presenter והמודל לא אמורות להיפגע מכך שמישתם את הממשק של View בצורה שונה.

כתיבת ה-GUI

במשימה זו עליכם לכתוב את ה GUI של הפרויקט.

- תבדילו בין המרכיבים "הסביבתיים" של החלון (כפתורים, תפריטים וכו') לבין לוח המשחק שהוא רכיב עצמאי המצורף לפרויקט שלנו בהתאם לתזכורת לעיל. נרצה שתהיה לנו היכולת לשלוף את לוח המשחק כיחידה אחת לפרויקט אחר, וכן להחליף את לוח המשחק בקלות בפרויקט שלנו.
- כמו כן, עלינו להגדיר את הפונקציונאליות של לוח המשחק מבחוץ, כך שנוכל להתאים אותה כרצוננו בכל פרויקט.
- אפשרו למשתמש לבקש יצירה של מבוך ע"פ קריטריונים משתנים.
- המבוך כמשחק לוח – עליכם לאפשר למשתמש לנסות לפתור את המבוך בעצמו ע"י הזזת הדמות באמצעות המקלדת. על מנת לאפשר למשתמש לנוע בכל הכיוונים (כולל אלכסונים) השתמשו ב NumPad:
 - בספרות 2,4,6,8 עבור שמאלה, ימינה, למעלה, למטה
 - בספרות 3,1,7,9 עבור אלכסונים בכיוונים עליון שמאלי, עליון ימני, תחתון שמאלי ותחתון ימני.
- הראו למשתמש בדרך יצירתית כלשהי שהוא הצליח לפתור את המבוך.
- אפשרו למשתמש לבקש פתרון למבוך שמוצג כרגע. הפתרון יוצג על גבי המבוך ויאפשר למשתמש להמשיך לשחק ולהיעזר בפתרון.
- אפשרו למשתמש לשמור את המבוך המוצג כרגע לקובץ בדיסק.
- אפשרו למשתמש לטעון מבוך ששמר בקובץ בעבר.

תוספות:

- צלילים:
 - הוסיפו מוזיקת רקע לאורך המשחק.
 - השמיעו מוזיקה שונה מתאימה כאשר המבוך נפתר.
- לחיצה על מקש Ctrl והזזת הגלגלת של העכבר תבצע zoom in / out ללוח המשחק.
 - שימו לב ששינוי גודל החלון של המשחק אמור לשנות את האלמנטים בתוך החלון.
- אפשרו למשתמש לגרור את הדמות על המסך באמצעות סמן העכבר (בנוסף לאפשרות של הזזתה ע"י מקשי המקלדת). הדמות חייבת לזוז בהתאם לתוואי הקירות (ולא לעבור דרכם) ואך ורק בדרכים האפשריות.

עיצוב ה-GUI

עצבו את חלון המשחק ואת כל מרכיביו כרצונכם, כל עוד אתם מקיימים את הדרישות הבאות:

1. אסור לקבע את גודל חלון המשחק. מותר למשתמש לשנות את גודלו. האלמנטים שבפנים צריכים להתאיין לגודל החלון.
2. צריך להיות תפריט עליון בחלון menu. תוכלו להכניס בו איזה אלמנטים שאתם רוצים אך המינימום הוא תפריט עם האלמנטים הבאים. קחו אלמנטים מאפליקציות שאתם מכירים. סדרו אותם לפי סדר הגיוני.

File .a

- .i New** – יצירת מבוך חדש.
- .ii Save** – שמירת המבוך הנוכחי לקובץ בדיסק.
- .iii Load** – טעינת מבוך מקובץ.

.b Options

- .i Properties** שפותח ומציג בצורה מסודרת את ההגדרות מתוך קובץ ההגדרות של האפליקציה. קובץ ההגדרות צריך לשבת בתוך הפרויקט הנוכחי (לא בתוך ה-Jar). רצוי לשים אותו בתוך תיקיית resources.
- .c Exit** – יגרום ליציאה מסודרת מהתוכנית ללא קבצים או ת'רדים פתוחים. שימו לב שניתן לצאת גם ע"י לחיצה על ה-X, גם אז היציאה צריכה להיות מסודרת. הקפידו שאין קוד כפול.
- .d Help** – נתוני עזר למשחק, כגון כללי המשחק, סימונים שונים על הלוח וכו'.
- .e About** – יכיל פרטים על המתכנתים, האלגוריתמים בהם אתם משתמשים ליצירת המבוך ולפתורו וכו'.

3. בלוח המשחק שלכם צריך לעשות שימוש בתמונה. לדוגמה הדמות במבוך, או הרקע. אין בעיה להשתמש בגרפיקה ובפקדים מותאמים אישית לרוב הדברים אך לפחות אלמנט אחד צריך להיות תמונה.

4. הודעות שגיאה יוצגו מעתה בחלון Alert.

5. תתפרעו! תוסיפו מה שבא לכם כל עוד לדעתכם זה ירשים את המשתמש, הבודק (או המראיין).

- **שימו לב** – יינתן לכם ניקוד יחסי על החלק של "הרשמת הבודק", כלומר, מי שיוסיף את הדברים הכי מרשימים לפרויקט שלו (לפי דעתו של הבודק) יקבל מלוא הנקודות על הסעיף הזה (10 נקודות מסך הנקודות של חלק ג של הפרויקט). שאר הסטודנטים יקבלו ציון יחסי על הסעיף הזה. חישבו איזה פיצ'רים אתם יכולים להוסיף לפרויקט שלכם על מנת להרשים את הבודק. ☺

דגשים:

1. מבנה הפרויקט:

a. תיקיית ה-src תכיל אך ורק את השכבות View, ViewModel, Model.

b. משאבים אחרים (תמונות, קבצי שמע, וכו'), ישבו בתיקיית resources שקיימת לצד תיקיית ה-src.

2. חשוב שלמשתמש שעובד עם המשחק שלכם יהיה ברור איך להפעיל משחק מבוך ואיך לשחק. ממשק שמכיל המון כפתורים יגרום למשתמש ללחוץ על כפתורים כדי לנסות לשחק, לפעמים הוא

יעשה זאת לא לפי הסדר שאתם חשבתם עליו מה שיכול לגרום שגיאות בזמן ריצה. תוכלו להשתמש ב-**Property disabled = true** של הכפתורים על מנת לנעול כפתורים שאתם לא רוצים שהמשתמש יקיש עליהם בשלב מסוים. תוכלו לפתוח את הכפתור ע"י **disabled = false** כרצונכם. לדוגמה, בטרם מוצג מבוך על המסך והמשתמש יכול לשחק, אין משמעות לכפתור Solve שפותח את המבוך. עדיף שהכפתור יהיה נעול מאשר יהיה פתוח ולחיצה עליו תביא לקריסה (במקרה הרע) או להודעה שלא ניתן לפתור את המבוך כי אין מבוך (במקרה הטוב).

3. כאשר אתם רוצים לקלוט מהמשתמש פרמטרים, לדוגמה עבור יצירת מבוך, תוכלו (אופציונאלי) לעשות זאת באמצעות חלון שיציג טופס ייעודי לכך. לאחר מילוי הטופס ולחיצה על הכפתור OK תוכלו לשלוף מתוך הטופס את הפרטים שהמשתמש הקיש. בנוסף, חשוב שתעשו בדיקות על הקלט ואם יש בעיה עם הקלט הציגו הודעה מתאימה למשתמש.

4. בדקו שאין מקרי קצה בהם האפליקציה קורסת.

5. השתמשו בקבצי מדיה רזים ככל שניתן, קבצים כבדים מדי יאטו לכם את האפליקציה וגם המשקל הכולל של האפליקציה יעלה. השתמשו בתמונות מסוג JPG, בקבצי סאונד מסוג Mp3 (אם ניתן) ובווידיאו בפורמט דחוס כגון MPEG. בחרו תמונות בגודל (פיקסלים) הקטן ביותר הנדרש עבור התצוגה שלכם, אין צורך להשתמש בתמונה בגודל 600 * 600 כאשר המטרה היא לצייר קיר במבוך

או את הדמות, יכולה להספיק לנו תמונה בגודל $100 * 100$.

משימה ג' – Maven & Log4j2 – 5 נקודות

- הוסיפו לפרויקט שלכם מודול Maven.
- טענו באמצעות Maven את ספריית Log4j2 כפי שהודגם בתרגול.
- הגדירו שהשרתים שלכם, שיוצרים ופותרים מבוכים, ירשמו לקובץ Log אינדיקציות מעניינות, כגון:
 - רישום של לקוח שפונה ומבקש ליצור מבוכ, פרטי הלקוח ונתוני המבוכ.
 - לקוח שפונה ומבקש לפתור מבוכ, פרטי הלקוח ונתוני האלגוריתם ששימש לפתרון המבוכ, ונתונים על הפתרון.
- הגדירו שקבצי הלוג יכתבו לתוך תיקייה שנקראת logs, לצד תיקיית src.
- השתמשו ברמות שונות של לוגינג בהתאם למידע (info, warning, error, fatal, ...).
- (אופציונאלי) להגדיר שכל שרת יכתוב לקובץ לוג נפרד משלו.

דגשים להגשה

1. הגישו את תיקיית הפרויקט שלכם במלואה.
2. הסירו קבצי Version שלא נחוצים להגשה, לדוגמה התיקייה Git. (תיקיה נסתרת).
3. בדקו שהפרויקט שלכם רץ באופן תקין על כל הפונקציונאליות שלו, גם במחשב אחר.
4. וודאו שכאשר צירפתם את קובץ ה-Jar מחלק ב' הוא נטען ממקום יחסי בפרויקט ולא מתוך מיקום אבסולוטי במחשב. הדבר חשוב ביותר מאחר והפרויקט צריך להטען גם במחשב אחר שבו הקובץ כבר לא נמצא במיקום האבסולוטי הישן. דרך ה- IntelliJ תוכלו ללחוץ מקש ימני על הפרויקט שלכם Open Module Settings (או F4). תחת Project Settings בחרו ב- Modules, כנסו לטאב Dependencies, כאן אתם צריכים לראות את ה-Jar שצירפתם לפרויקט. אם לצידו רשום גם את המיקום שלו (מיקום אבסולוטי במחשב) תוכלו ללחוץ עליו מקש ימני ולבחור Move to project libraries.

בהצלחה!